

Module 1: Lesson 2

Introduction to Machine Learning



Outline

- ▶ Linear Models and regularization
- ▶ Hyperparameter tuning

Linear models and overfitting

Regularization is a good option for reducing the overfitting of an ML model. Regularization means constraining the learned parameters θ and reducing the degrees of freedom of the model to fit the training data.

In linear regression, we can regularize the parameters by minimizing in training a general cost function of the form:

$$J(\theta) = MSE(\theta) + Penalty(\theta)$$

We next describe three methods: Ridge, Lasso, and Elastic Net. Each method specifies different $Penalty(\theta)$ functions to constrain the parameters.

► **Important:** The penalty term should only be added to the cost function during training.

In the validation or test samples, we are only interested in evaluating the unregularized performance of the model, say, the $MSE(\theta)$ in linear regression.

The methods described below are of general application in ML learning problems and are not specific to the Linear Regression framework.

Ridge Regression

Ridge Regression is a *shrinkage* regularization technique that aims at keeping the parameter values of the model as small as possible. The penalty function is of the form:

$$\alpha \sum_{i=1}^n \theta_i^2$$

Notice that we do not aim to regularize the bias term θ_0 . Ridge Regression “forces” the model to become closer to a model where we use the average in training as the prediction.

The hyperparameter α determines how much we wish to regularize the model. A very high α will fit the data with a flat line through the average of the labels y .

Using Gradient Descent with Ridge Regression, the update of the parameters will be given by:

$$\theta^{[next\ step]} = \theta^{[previous\ step]} - \eta \nabla_{\theta} MSE \left(\theta^{[previous\ step]} \right) - a \odot \theta^{[previous\ step]}$$

where a is a column vector of n elements all of which have the value α , except the first term that has a value of 0 and $\odot$ refers to element-by-element multiplication.

Lasso Regression

The Lasso (Least Absolute Shrinkage and Selection Operator) Regression sets to zero the parameters associated with the least important features. That is, Lasso Regression performs a selection of features that are useful for prediction. The regularization term is of the form:

$$\alpha \sum_{i=1}^n |\theta_i|$$

The Lasso penalty term is an ℓ_1 -norm of the parameter vector, while the Ridge version uses an ℓ_2 -norm.

The Lasso term is not differentiable at $\theta_i = 0$, so the Gradient Descent algorithm needs to be adjusted properly using a *subgradient* method:

$$\theta^{[next\ step]} = \theta^{[previous\ step]} - \eta \nabla_{\theta} MSE \left(\theta^{[previous\ step]} \right) - a \odot sign(\theta)$$

where $sign(\theta)$ takes a value of -1 for those elements where the parameters are negative, 1 where the parameters are positive, and 0 where the parameters are strictly equal to 0.

Elastic Net Regression

The Elastic Net combines Ridge and Lasso regularization, where a hyperparameter determines the combination of shrinkage and feature selection.

The Elastic Net penalty is given by:

$$r\alpha \sum_{i=1}^n |\theta_i| + \frac{1-r}{2}\alpha \sum_{i=1}^n \theta_i^2$$

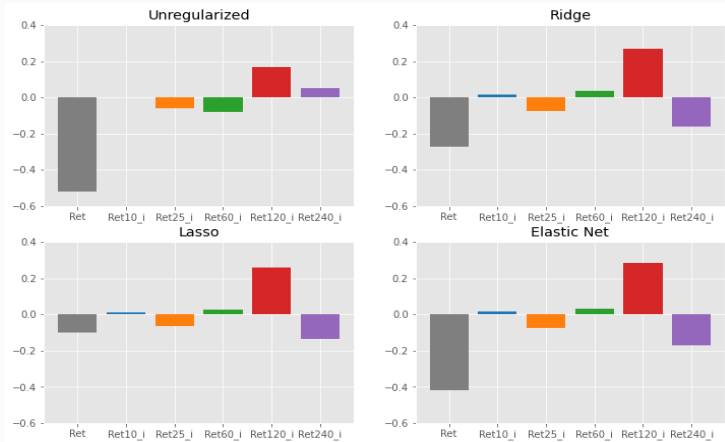
α determines the importance of regularization, r balances the mix of feature selection and shrinkage.

In an Elastic Net Regression with Gradient Descent, we would have, at least, three hyperparameters:

- ▶ Learning rate η .
- ▶ Overall regularization of the parameters α .
- ▶ Relative weight of Lasso vs. Ridge regularization r .

Below, we devise a method to obtain "good" values of those hyperparameters.

Time Series Momentum: Results



Hyperparameter tuning: Cross-validation

We may opt for a manual, *brute force* approach to check the best values for the hyperparameters. This would be quite inefficient because we may not have time to explore many combinations.

A more efficient technique exploits cross-validation: **k-fold cross-validation**.

In this method, the training set is split into k complementary subsets or “folds.” For each fold, the model is trained against the remaining folds using different hyperparameters, where the fold is used as a validation set.

Once the model type and hyperparameters have been selected, a final model is trained using these hyperparameters on the full training set, and the generalized error is measured on the test set.

Repeated k-fold cross-validation: We repeat the k-fold cross-validation process multiple times to avoid the noisiness from a small value of k .

Hyperparameter tuning: k-fold cross-validation illustration



Note: In all plots, the x-axis represents the predicted return from the models. The y-axis represents the actual return.

Hyperparameter tuning: Other tuners

The grid search approach is fine when you are exploring relatively few combinations, but it may get computationally costly when the hyperparameter search space is large.

Other alternatives for hyperparameter tuning:

- ▶ **Randomized search:** it evaluates random combinations of hyperparameters by selecting a random value for each hyperparameter at every iteration. Explore different values of the hyperparameters at each run, instead of pre-determined ones.
- ▶ **Ensemble methods:** allows for combining several predictors from different trained models to reach an even better predictor, combining the models that perform best.

Summary of Lesson 2

In Lesson 2, we have looked at:

- ▶ Methods to regularize ML models
- ▶ Methods to fine-tune the hyperparameters

⇒ **References for this lesson:**

Géron, Aurélien. *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow*. 2nd ed., O'Reilly Media, Inc., 2019.

TO DO NEXT: Now, please go to the associated Jupyter Notebook for this lesson to further explore the concepts covered in these slides and the application for our time series Momentum strategy.

In the next lesson, we will review the main concepts of ML classification problems.